ShopPulse

De la donnée brute à la décision métier

Plateforme Data & IA · dataset Olist · Mission Flash (rattrapage HETIC)

98 199 commandes. 8,1 % arrivent en retard.
Et un retard coûte 1,7 point de note au client (2,57 contre 4,29).

RFERT · github.com/RFERT/ShopPulse_Data_Consulting

Le problème et la valeur apportée

Trois irritants, trois leviers

Direction

Irritant : pas de vue unique sur l'activité.

Levier : dashboard de 5 KPI, alimenté par une base consolidée.

Marketing

Irritant : clients jamais segmentés, peu de réachat.

Levier : segmentation RFM, 3 personas, 3 campagnes chiffrées.

Logistique & SAV

Irritant : retards et avis négatifs subis.

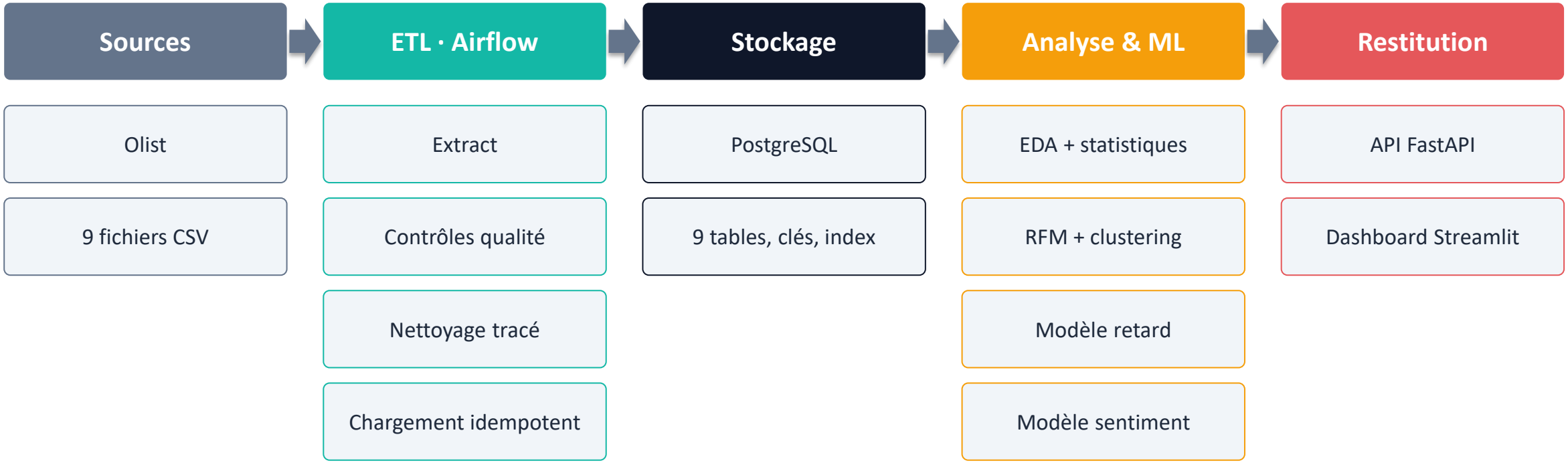
Levier : prédiction du retard et analyse de sentiment, exposées en API.

Test de Mann-Whitney : note 2,57 (retard) contre 4,29 (à l'heure), p ≈ 0.

Le retard est le premier levier de satisfaction.

Architecture cible

Du fichier source à la décision, entièrement conteneurisé



Arbitrages techniques

Coût · délai · adéquation · pérennité

Besoin	Choix retenu	Alternative écartée	Pourquoi
Orchestration ETL	**Airflow**	Prefect	Standard industriel, UI de supervision, compétence transférable
Restitution	**Streamlit**	Metabase	100 % Python, 5 KPI sur mesure, une seule stack à maintenir
Avis en portugais	**Entraînement direct**	Traduction	Pas de coût ni de perte d'information, pipeline simple
API	**FastAPI**	Flask	Validation Pydantic et OpenAPI natives

Le pipeline reste découplé d'Airflow (exécutable en CLI) : réversibilité préservée.

Les données : pipeline et qualité

Rejouable, traçable, idempotent



9

tables reliées par clés étrangères

≈ 1,55 M

lignes chargées dans PostgreSQL

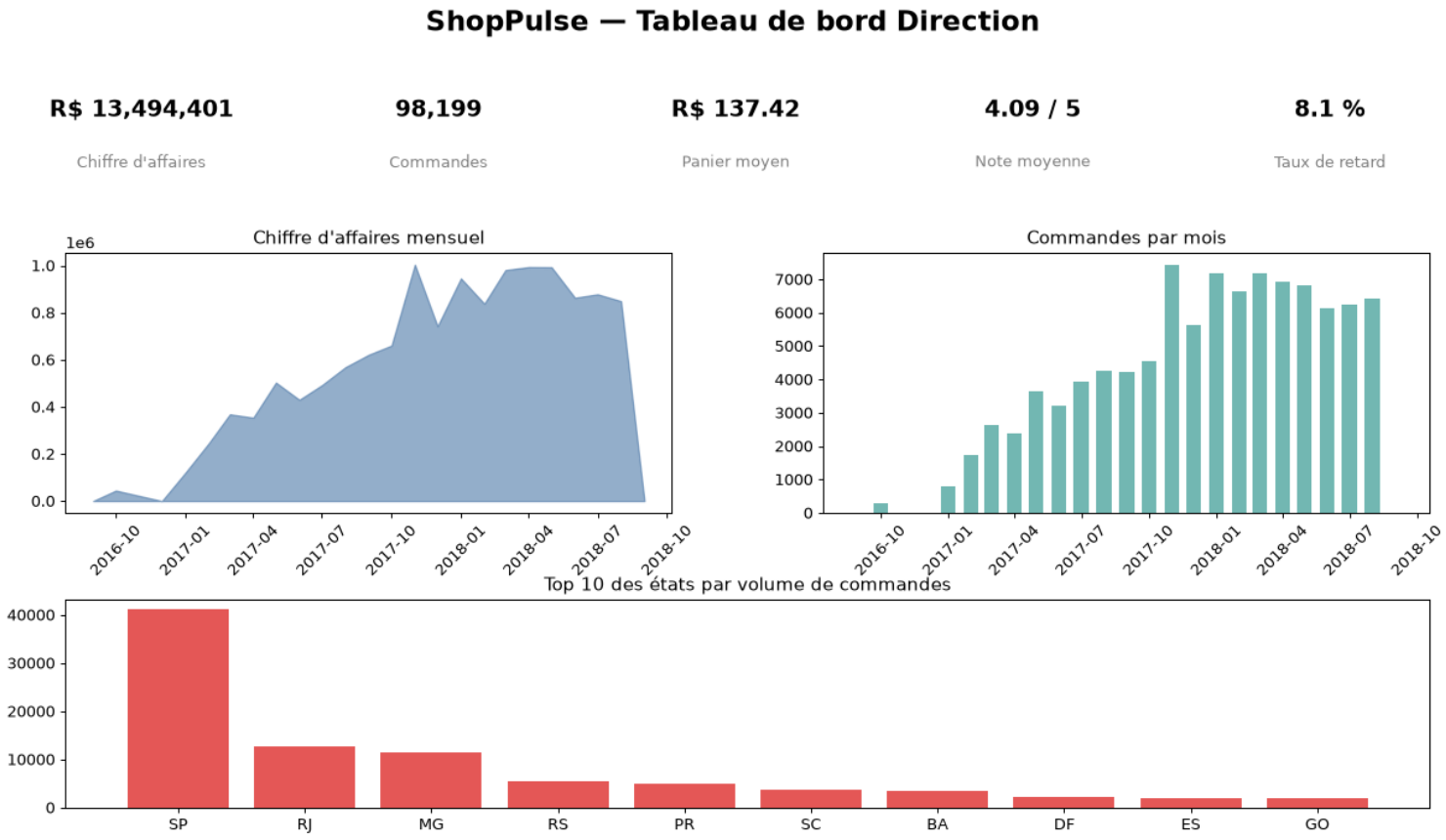
0

ligne supprimée au nettoyage

- Rapport de qualité avant / après généré automatiquement (lignes, nulls, doublons).
- Règles de nettoyage tracées : ex. 13 catégories produit inconnues mises à NULL.
- 261 831 doublons de géolocalisation repérés et documentés.
- Rejouer le pipeline produit exactement le même état.

Réponse métier n°1 : le tableau de bord Direction

5 KPI, une seule page



R$ 13,49 M

Chiffre d'affaires

98 199

Commandes

R$ 137,42

Panier moyen

4,09 / 5

Note moyenne

8,1 %

Taux de retard

São Paulo concentre l'essentiel des commandes ; un retard de 8,1 % pèse sur la note.

Réponse métier n°2 : segmentation RFM et recommandations

Clustering sur Récence, Fréquence, Montant · ~95 000 clients

Victor · 35 %

Nouvel acheteur à fort potentiel

~33 200 clients, panier R$ 251, un seul achat.

Relance cross-sell J+30 (5 % de réachat).

~R$ 417 000

Lucia · 29 %

Cliente dormante

~27 500 clients, dernier achat il y a ~476 jours.

Win-back à -15 % (3 % de réactivation).

~R$ 90 000

Marco · 3 %

Fidèle premium

~2 900 clients, 2,1 commandes, R$ 260.

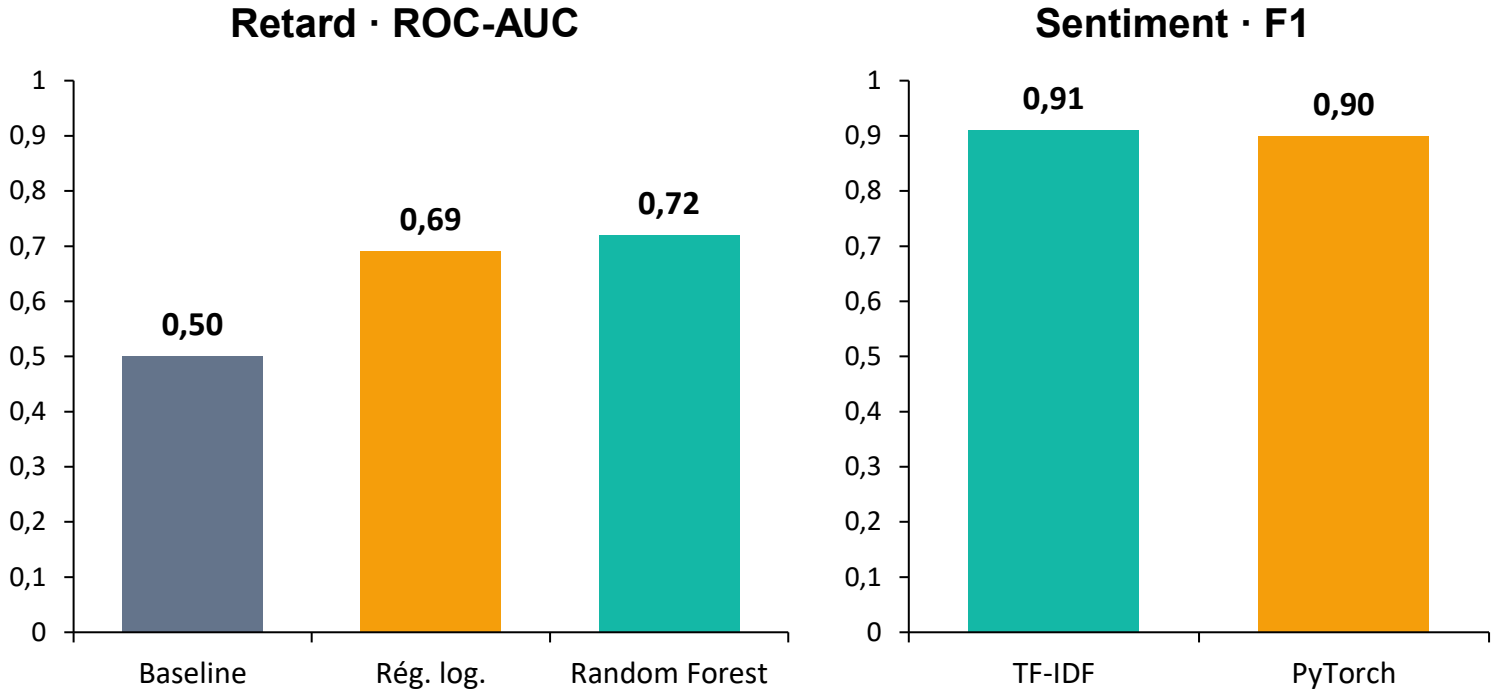
Programme VIP (+0,3 commande par an).

~R$ 226 000

≈ R$ 733 000 de CA incrémental estimé (hypothèses prudentes, voir docs/marketing.md)

Les modèles : performances, limites et biais

Validation croisée 5 plis, comparaison à une baseline



Limites et biais

- Rappel faible à 0,5 : l'API expose la probabilité, seuil métier 0,30.
- Audit d'équité par région : écart de rappel 0,86 (élevé à Rio, bas dans le RS).
- Corrections prévues : seuils par région, calibration.

Le retard reste difficile à prédire : le modèle sert à prioriser, pas à décider seul.

Le deep learning est battu par une régression logistique : la baseline reste le juge de paix.

Démonstration : l'API conteneurisée

En direct : docker compose up, puis deux appels

```
$ docker compose -f infra/docker-compose.yml up -d

$ curl -X POST localhost:8000/predict/sentiment \
    -d '{"texte": "Produto ótimo, chegou rápido!"}'
{"sentiment":"positif","probabilite_positif":0.97}

$ curl -X POST localhost:8000/predict/retard \
    -d '{... commande SP -> RJ ...}'
{"probabilite_retard":0.73,
 "risque_eleve":true,"seuil":0.3}
```

Pourquoi une API ?

Un endpoint par modèle, entrées validées (Pydantic), documentation OpenAPI sur /docs.

Prêt à tourner

Image Docker unique, configuration par variables d'environnement, ports modifiables.

Bilan : écarts, bonus et suite

Ce qui est livré, et ce que je referais

Bonus réalisés

- MLflow : suivi des expériences
- ELK : logs JSON et supervision
- Spark : analyse du clickstream REES46
- AWS : plan ECS Fargate + RDS, ~50 USD/mois (non déployé)

Qualité

- 34 tests, couverture 76,8 %
- ruff + GitHub Actions : intégration continue verte
- Un tiers clone et lance en moins de 30 minutes

Ce que je referais

- Meilleur rappel sur le retard, seuils par région
- Contrôles de données (Great Expectations), suivi de dérive

Transparence

- L'IA a été utilisée comme assistant de développement.
- Usage détaillé dans le README (section « Usage de l'IA »).